

Election du Chef (Leader election)

Master informatique
UE AR (4I403)

Plan

2

- I. Introduction
 - 1. Principe
 - 2. Applications
 - 3. Impossibilité
- II. Topologie en anneau
- III. Topologie quelconque avec hypothèses

MU4IN403 – Election de chef

I.1 Principe

3

Objectif

Désigner un chef dans un ensemble de processus

Propriétés

- 1. Sûreté
Le chef doit être unique
- 2. Vivacité
La désignation doit se faire en un temps fini

I.2 Applications

4

- Mise en place d'un contrôle centralisé (coordinateur, séquenceur, etc.)
- Recouvrement de défaillance
 - Restauration d'un maître dans un système maître/esclaves
 - Régénération de jeton perdu
 - Serveurs répliqués. Le serveur primaire est le leader.
- Consensus
- Résolution de situations de blocage

Election chef x exclusion mutuelle

5

Election chef

- Processus candidat peut donner la main à un autre processus.
- Vivacité : la désignation doit se faire en un temps fini
- L'identifiant du nouveau chef élu est annoncé aux autres processus

Exclusion mutuelle

- Processus demandeur est en concurrence avec les autres demandeurs jusqu'à ce qu'il réussisse
- Vivacité : toute requête doit être satisfaite à terme.
- Les processus ne soucient pas de connaître l'identité du processus en SC.

MU4IN403 – Election de chef

I.3 Impossibilité

6

[Angluin 1980]

Il n'existe pas d'algorithme déterministe d'élection de chef dans les réseaux anonymes et uniformes.

Idée de la preuve

1. Réseau anonyme $\Rightarrow$ config. de départ peut être symétrique
2. Config. objectif (leader unique élu) est asymétrique
3. Il existe une exécution E du système telle que
à partir d'une configuration symétrique
on passe **toujours** dans une configuration symétrique

MU4IN403 – Election de chef

I.3 Impossibilité

7

Contourner l'impossibilité

- via des identifiants
Relation d'ordre entre identifiants brise la symétrie
- via des algorithmes probabilistes
 - Tirages aléatoires sur chaque nœud
 - Tomber à l'infini sur des configurations symétriques est quasi-impossible
 - Algorithmes Las Vegas : algorithme probabiliste
 - S'arrête toujours avec une réponse exacte.
 - Toutes les configurations terminales sont correctes.

MU4IN403 – Election de chef

Plan

8

- I. Introduction
- II. Topologie en anneau
 1. Anneau unidirectionnel asynchrone (Chang-Roberts)
 2. Anneau unidirectionnel synchrone anonyme (Itai-Rodeh)
 3. Anneau bidirectionnel asynchrone (Hirschberg-Sinclair)
- III. Topologie quelconque avec hypothèses

MU4IN403 – Election de chef

II.1 Unidirectionnel synchrone

9

[Chang & Roberts 1979] (adapté de LeLann77)

Hypothèses

- Les nœuds sont capables de s'organiser en anneau unidirectionnel
- Chaque nœud possède un identifiant unique dans l'anneau
- Chaque nœud i dispose d'une référence $\text{succ}[i]$ vers son successeur
- Aucun nœud ne connaît la taille de l'anneau
- Plusieurs candidats simultanés possibles
- Communications fiables asynchrones FIFO
- Exécution sans faute

Idée

- Chaque candidat propage sa candidature dans l'anneau
- Un candidat qui reçoit une candidature supérieure a perdu
- Le processus candidat d'identifiant maximal reçoit sa propre candidature

MU4IN403 – Election de chef

II.1 Unidirectionnel synchrone

10

[Chang & Roberts 1979] - Algorithme

Variables locales

- Etat – non-candidat/candidat/élu/perdu, initialisé à non-candidat
- leader – identité du leader, initialisé à NULL
- $\text{succ}[i]$ – successeur de i dans l'anneau

Deux types de message

ELEC et LEADER

Site i se porte candidat

```
Proposer_candidature ( ) {  
    Etat = candidat  
    leader = i  
    envoyer(<ELEC, i>) à  $\text{succ}[i]$   
}
```

MU4IN403 – Election de chef

II.1 Unidirectionnel synchrone

11

[Chang & Roberts 1979] – Algorithme (suite)

Site i reçoit <ELEC, j >

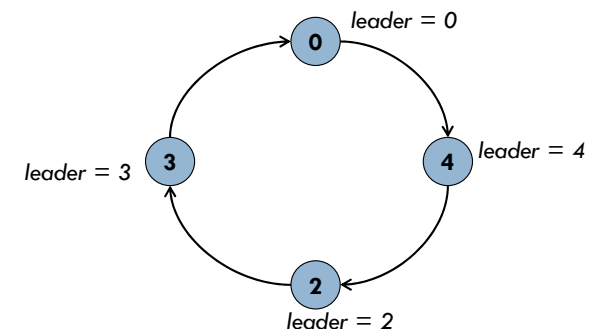
- ```
if $i > j$ then
 if Etat = non-candidat then
 Proposer_candidature ()
else if $i < j$ then
 Etat = perdu
 leader = j
 envoyer (<ELEC, j >) à $\text{succ}[i]$
else if $i = j$ then
 Etat = élu
 envoyer (<LEADER, i >) à $\text{succ}[i]$
```

MU4IN403 – Election de chef

## II.1 Unidirectionnel synchrone

12

[Chang & Roberts 1979] - Exemple

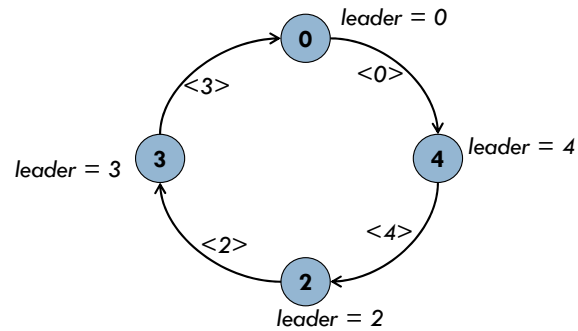


MU4IN403 – Election de chef

## II.1 Unidirectionnel synchrone

13

### [Chang & Roberts 1979] - Exemple

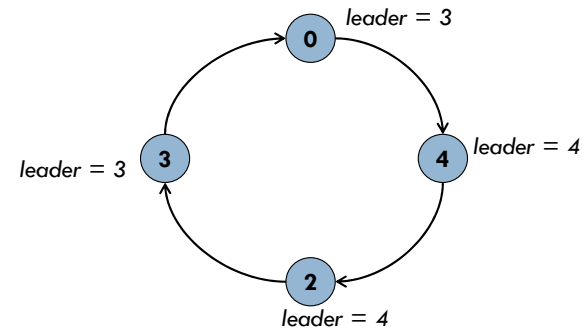


MU4IN403 – Election de chef

## II.1 Unidirectionnel synchrone

14

### [Chang & Roberts 1979] - Exemple

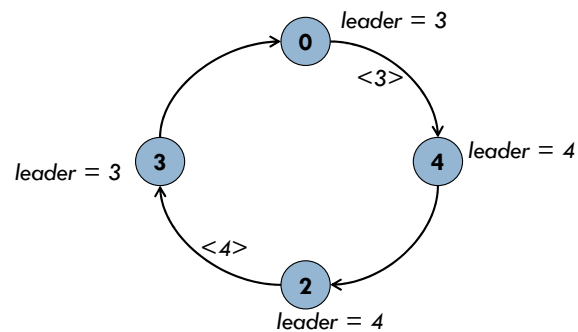


MU4IN403 – Election de chef

## II.1 Unidirectionnel synchrone

15

### [Chang & Roberts 1979] - Exemple

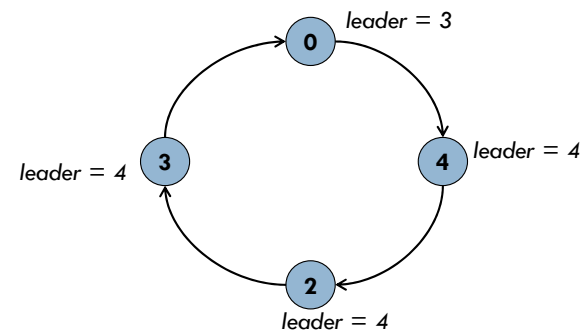


MU4IN403 – Election de chef

## II.1 Unidirectionnel synchrone

16

### [Chang & Roberts 1979] - Exemple

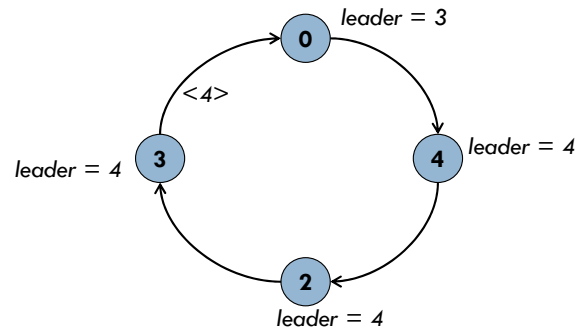


MU4IN403 – Election de chef

## II.1 Unidirectionnel synchrone

17

### [Chang & Roberts 1979] - Exemple

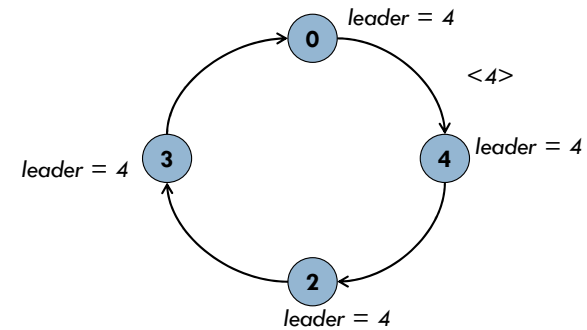


MU4IN403 – Election de chef

## II.1 Unidirectionnel synchrone

18

### [Chang & Roberts 1979] - Exemple



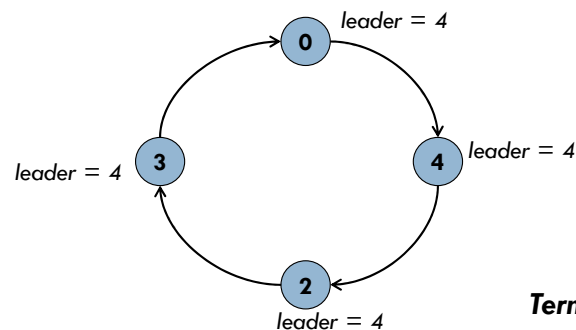
Election terminée, le nouveau leader fait l'annonce : message LEADER avec son ID

MU4IN403 – Election de chef

## II.1 Unidirectionnel synchrone

19

### [Chang & Roberts 1979] - Exemple



**Terminaison**

Election terminée, le nouveau leader fait l'annonce : message LEADER avec son ID

MU4IN403 – Election de chef

## II.1 Unidirectionnel synchrone

20

### [Chang & Roberts 1979] – Complexité

Complexité en temps :  $O(N)$

Complexité en memoire :  $O(\log N)$

MU4IN403 – Election de chef

## II.1 Unidirectionnel synchrone

21

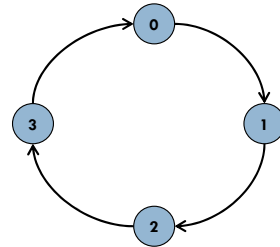
### [Chang & Roberts 1979] – Complexité

Complexité en temps :  $O(N)$

Complexité en memoire :  $O(\log N)$

Complexité en messages :

Meilleur des cas :  $O(N)$



MU4IN403 – Election de chef

## II.1 Unidirectionnel synchrone

22

### [Chang & Roberts 1979] – Complexité

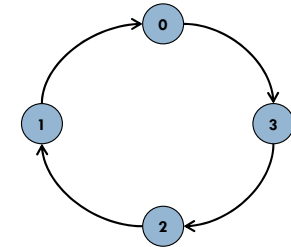
Complexité en temps :  $O(N)$

Complexité en memoire :  $O(\log N)$

Complexité en messages :

Meilleur des cas :  $O(N)$

Pire des cas :  $O(N^2)$



MU4IN403 – Election de chef

## II.1 Unidirectionnel synchrone

23

### □ Pire cas complexité de message

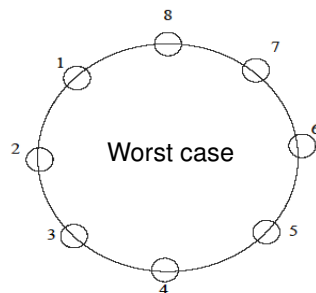
- tous les processus se portent candidats

ELEC 1 : 1 lien

ELEC 2 : 2 liens

...

ELEC 8 : 8 liens



$$N + (N-1) + \dots + 1 =$$

$$\sum_{j=1}^N j = N(1+N)/2 = O(N^2)$$

Message ELEC de 1 fait 1 saut; message ELEC de 2 fait 2 sauts, ...

### □ Meilleur cas : que node 8 démarre l'élection : $O(N)$ .

MU4IN403 – Election de chef

23

## II.2 Unidirectionnel synchrone anonyme

24

### [Itai & Rodeh 81]

Basé sur l'algorithme de Chang et Roberts

### Hypothèses

Les nœuds sont capables de s'organiser en anneau unidirectionnel

~~Chaque nœud possède un identifiant unique dans l'anneau~~

Chaque nœud  $i$  dispose d'une référence  $\text{succ}[i]$  vers son successeur

**Chaque nœud connaît la taille  $N$  de l'anneau**

Plusieurs candidats simultanés possibles

Communications fiables et synchrones

Exécution sans faute

MU4IN403 – Election de chef

## II.2 Unidirectionnel synchrone anonyme

25

### [Itai & Rodeh 81]

Initialement, tous les processus sont actifs

Chaque processus maintient état (init. actif) et tournoi (init. 1)

Fonctionnement en tournois entre nœuds actifs

Chaque nœud actif tire un identifiant **id** aléatoirement dans  $[1, k]$  ( $k \geq N$ ) puis envoie  $(id, 1, 1, \text{vrai})$  au successeur

Lorsqu'un processus  $p$  reçoit  $(\#id, \#tournoi, \#saut, \text{unique})$

Si  $p$  est inactive, alors il envoie  $(\#id, \#tournoi, \#saut+1, \text{unique})$  au successeur

sinon //  $p$  est active alors

si  $\#saut = N$  alors // je suis l'initiateur du tournoi

alors si unique alors état = élu

sinon tournoi = tournoi + 1

$p$  tire un nouvel **id** aléatoirement dans  $[1, k]$  puis envoie  $(id, \text{tournoi}, 1, \text{vrai})$  au successeur

sinon //  $\#saut \neq N$

si  $(id, \text{tournoi}) =_{\text{lex}} (\#id, \#tournoi)$  alors  $p$  envoie  $(\#id, \text{tournoi}, \#saut+1, \text{faux})$  au successeur

sinon

si  $(id, \text{tournoi}) <_{\text{lex}} (\#id, \#tournoi)$  alors

état = inactive;

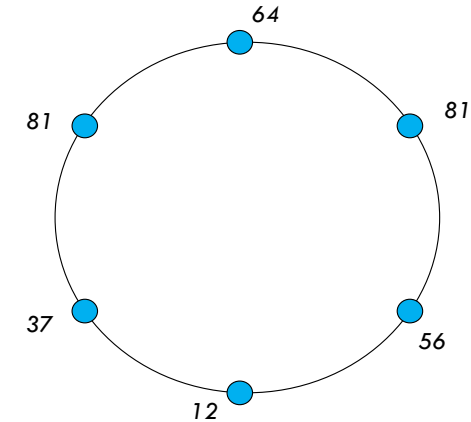
$p$  envoie  $(\#id, \text{tournoi}, \#saut+1, \text{unique})$  au successeur

MU4IN403 – Election de chef

## II.2 Unidirectionnel synchrone anonyme

26

### [Itai & Rodeh 81] - Exemple

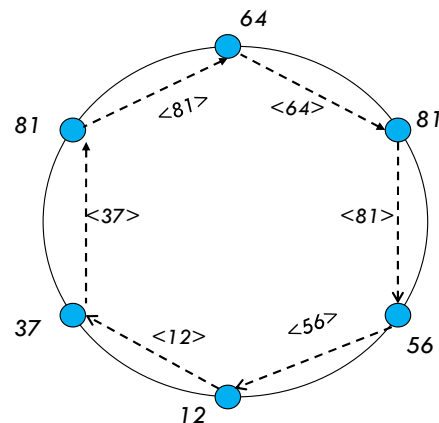


MU4IN403 – Election de chef

## II.2 Unidirectionnel synchrone anonyme

27

### [Itai & Rodeh 81] - Exemple

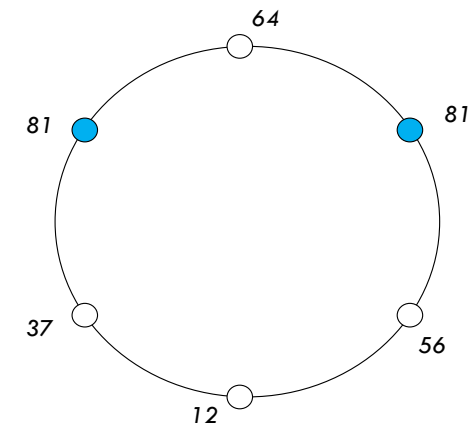


MU4IN403 – Election de chef

## II.2 Unidirectionnel synchrone anonyme

28

### [Itai & Rodeh 81] - Exemple

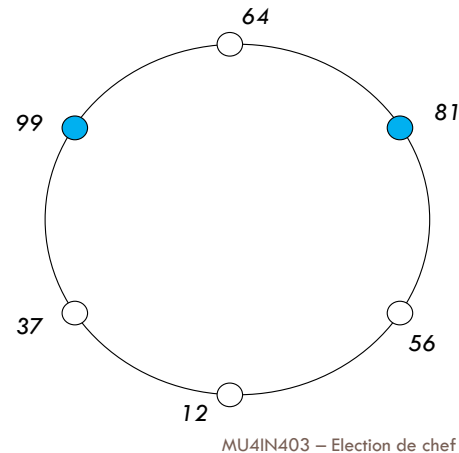


MU4IN403 – Election de chef

## II.2 Unidirectionnel synchrone anonyme

29

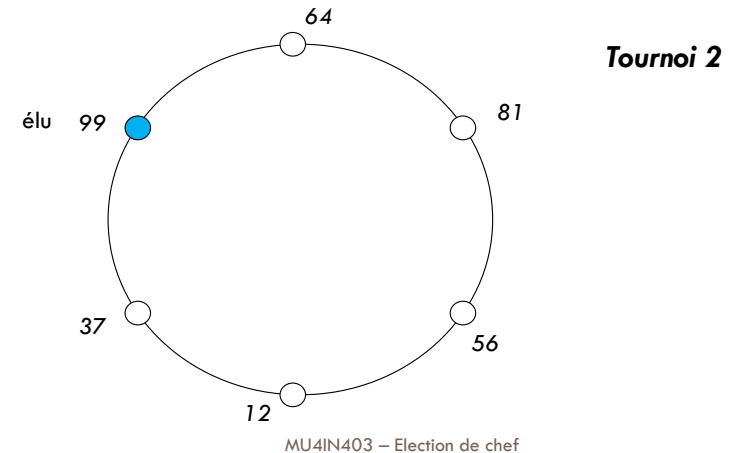
### [Itai & Rodeh 81] - Exemple



## II.2 Unidirectionnel synchrone anonyme

30

### [Itai & Rodeh 81] - Exemple



## II.2 Unidirectionnel synchrone anonyme

31

### [Itai & Rodeh 81] - Analyse

C'est un algorithme Las Vegas

Termine toujours

Cas moyen

Forte probabilité d'un seul candidat à chaque phase

Complexité en nombre de messages est  $O(N \log N)$

$\log N$  : nombre moyen de tournois.

## II.3 Bidirectionnel asynchrone

32

### [Hirschberg & Sinclair 1980]

#### Hypothèses

Les nœuds sont capables de s'organiser en anneau bidirectionnel

Chaque nœud possède

un identifiant unique dans l'anneau

une référence  $\text{succ}[i]$  vers son successeur

une référence  $\text{pred}[i]$  vers son prédécesseur

Aucun nœud ne connaît la taille de l'anneau

Plusieurs candidats simultanés possibles

Communications fiables et asynchrones

Exécution sans faute



## II.3 Bidirectionnel asynchrone

33

### [Hirschberg & Sinclair 1980] - Idée

- Fonctionnement en rondes asynchrones
  - ▢ Les nœuds *actifs* sont candidats.
    - Un nœud *actif* émet son identifiant  $id$  dans les 2 directions sur des distances croissantes
      - Ronde  $k$ : distance  $2^k$ ,  $k=0,1,2,\dots$  (distance 1,2,4,8,...)
    - Le vainqueur de chaque ronde est celui qui a transmis sur toute la distance
      - Il continue *actif*. Les autres passent à l'état *inactif*.
  - ▢ Un processus dont l'identifiant  $id$  a fait le tour de l'anneau est désigné le leader

MU4IN403 – Election de chef

## Hirschberg & Sinclair - Principe

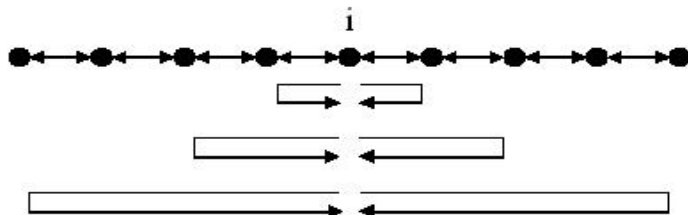
34

- Lorsque  $P_i$  reçoit un message avec  $id_j$ , il compare celui-ci avec son  $id_i$ .
- Si  $id_j < id_i$ ,  $P_i$  ne repasse pas le message
- Si  $id_j > id_i$ ,
  - ▢  $P_i$  devient *inactif*
  - ▢ si ce n'est pas la fin du chemin du message,
    - $P_i$  repasse le message à son successeur
  - ▢ sinon
    - $P_i$  repasse le message à son prédécesseur
- Si  $id_j = id_i$ , et  $id_j$  a fait le tour
  - ▢  $P_i$  se déclare leader

MU4IN403 – Election de chef

## Hirschberg & Sinclair

35



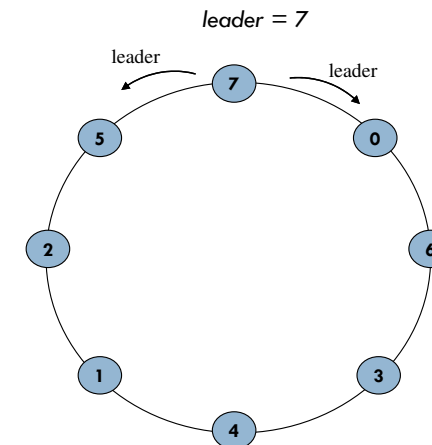
- complexité en messages :  $O(N \log N)$ 
  - ▢  $\log N$  = nombre de rondes
- Sera étudié en TD (algorithme et complexité)

MU4IN403 – Election de chef

## Hirschberg & Sinclair

36

### [Hirschberg & Sinclair 1980] – Exemple (node 7)



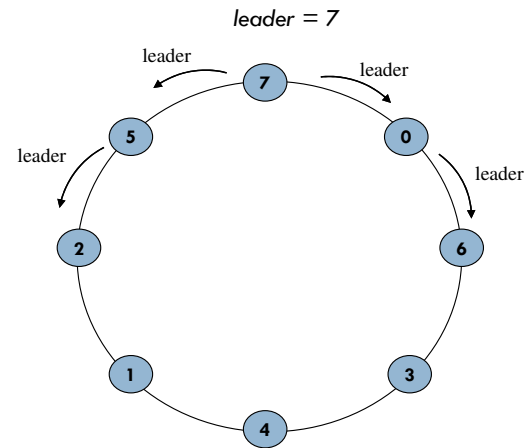
Ronde 0 – init  
 $2^k=1$

MU4IN403 – Election de chef

## Hirschberg & Sinclair

37

### [Hirschberg & Sinclair 1980] – Exemple (node 7)



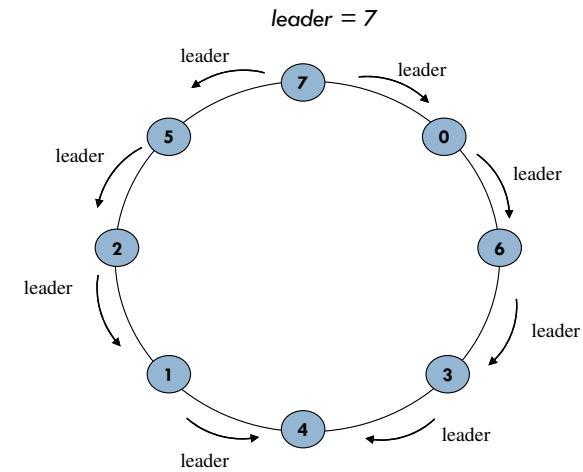
Ronde  $k=1$   
 $2^k=2$

MU4IN403 – Election de chef

## Hirschberg & Sinclair

38

### [Hirschberg & Sinclair 1980] – Exemple (node 7)



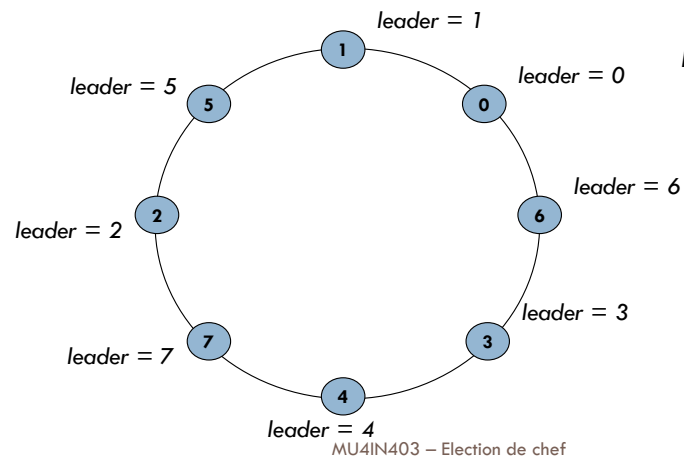
Ronde  $k=2$   
 $2^k=4$

MU4IN403 – Election de chef

## Hirschberg & Sinclair

39

### [Hirschberg & Sinclair 1980] – Exemple



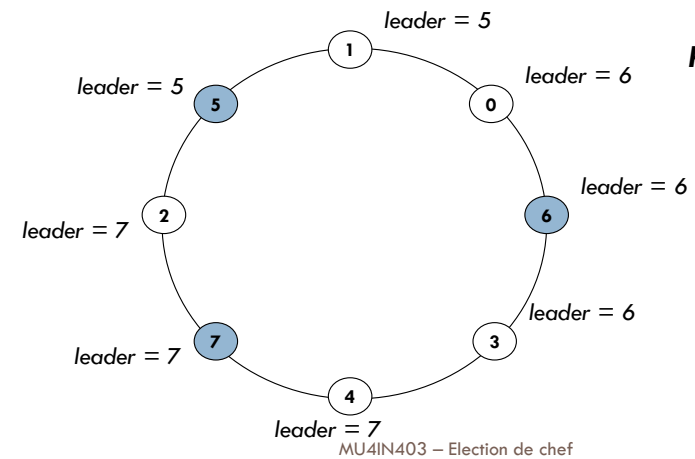
Ronde 0 – init

MU4IN403 – Election de chef

## Hirschberg & Sinclair

40

### [Hirschberg & Sinclair 1980] – Exemple



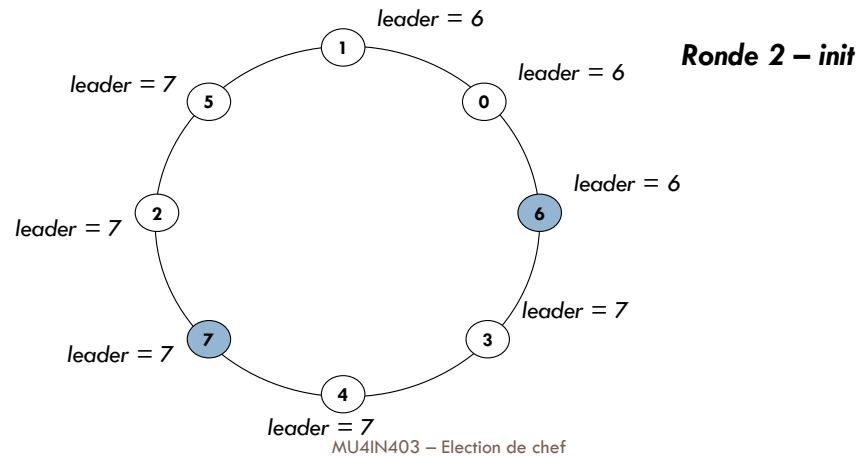
Ronde 1 – init

MU4IN403 – Election de chef

# Hirschberg & Sinclair

41

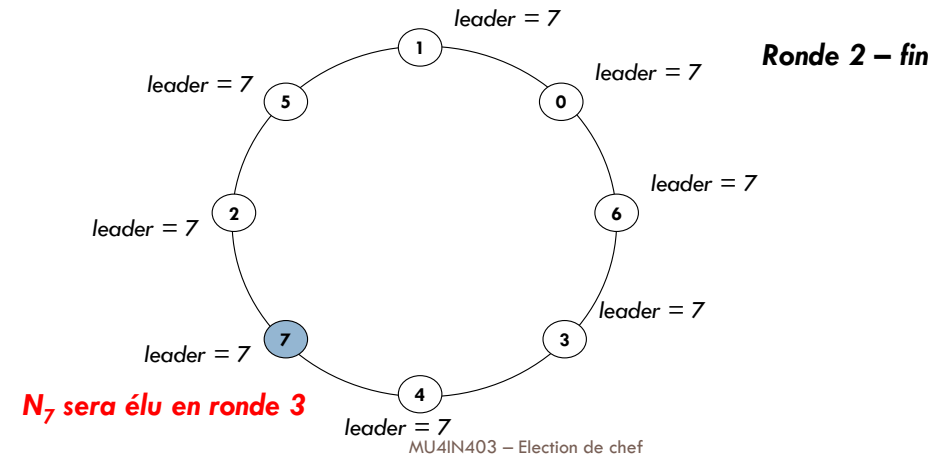
## [Hirschberg & Sinclair 1980] – Exemple



# Hirschberg & Sinclair

42

## [Hirschberg & Sinclair 1980] – Exemple



## Plan

43

- I. Introduction
- II. Topologie en anneau
- III. Topologie quelconque avec hypothèses
  1. Diffusion avec identifiants
  2. Diffusion anonyme

## III.2 Diffusion avec identifiants

44

### Calcul de maximum

#### Hypothèses

- Processus avec identifiants distincts
- Taille du réseau connu
- Accès à un broadcast
- Communications fiables et synchrones

#### Idée

Diffusion de son identité et attente des identités de tous les autres  
Le processus d'identifiant max devient le chef

Complexité en nombre de messages est  $O(M)$  dans tous les cas

$M$  = nombre de liens

$[O(NM)$  si initiateurs multiples  $\rightarrow O(N^3)]$  (réseau complet  $M = N^2$  liens)

## III.2 Diffusion avec identifiants

45

### FloodMax - Calcul de maximum par propagation

Algorithme par Vague (Total) de Phase

#### Hypothèses

- ▣ Processus avec identifiants distincts
- ▣ Diamètre du réseau D connu de tous
- ▣ Communications fiables et synchrones
- ▣ Graphe orienté fortement connexe

MU4IN403 – Election de chef

## III.2 Diffusion avec identifiants

46

#### Idée

- ▣ Valeur *leader* initialisée avec l'identifiant local du processus
  - ▣ Chaque processus diffusera sa valeur *leader* à ses voisins D fois
  - ▣ Un processus n'a le droit d'émettre un message à tous ses voisins sortants pour la  $(i+1)^{\text{ème}}$  fois qu'après avoir reçu le  $i^{\text{ème}}$  message de tous ses voisins entrants.
  - ▣ Sur réception de *leader'*,  $\text{leader} = \max(\text{leader}, \text{leader}')$
  - ▣ Après D phases, tout le monde connaît la valeur maximale pour *leader*
- ▣ Complexité en nombre de messages est  $O(D.M)$  avec M le nombre de liens
- ▣ Algorithme de Phase (Algorithme à Vague) vu en cours, TD et TME.

MU4IN403 – Election de chef

## III.2 Diffusion avec identifiants

47

### Bully [Garcia-Molina 1982]

#### Hypothèses

- ▣ Processus avec identifiants distincts
- ▣ Graphe complet
- ▣ Communications fiables et synchrones
- ▣ Pannes franches
- ▣ Trois types de message : ELEC, OK, LEADER

Idée : élire le processus non fautif dont l'identifiant le plus important

- ▣ Chaque candidat *i* envoie un message ELEC avec son identité aux processus dont l'identité est supérieure à la sienne.
  - ▣ Si après un délai (timeout) *i* n'a pas reçu aucune réponse (msg OK), il se déclare le chef.
  - ▣ Il envoie alors un message LEADER à tous les processus dont l'identité est inférieure à la sienne.
- ▣ En recevant ELEC un site répond avec le message OK et devient candidat, s'il n'en est pas encore.

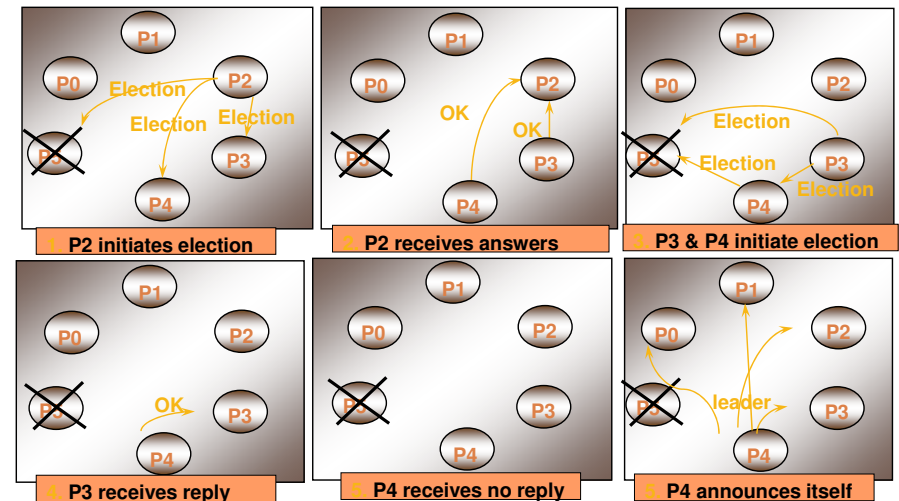
Complexité en nombre de messages est  $O(N^2)$  dans le pire des cas

Si tous les N-1 sites se portent candidat.

MU4IN403 – Election de chef

## III.2 Diffusion avec identifiants

48



## III.3 Diffusion anonyme

49

### Candidature aléatoire

#### Hypothèses

- Nombre total de nœuds connu de tous
- Accès à un broadcast
- Communications fiables et **synchrones**
- Pannes franches

#### Idée

Algorithme par phases, avec tous les nœuds initialement actifs

1. Chaque nœud *actif* décide aléatoirement s'il candidate
2. Un nœud *actif* diffuse sa (non-)candidature et attend des réponses
3. Si un seul processus candidate il devient le chef
4. Sinon phase suivante

Complexité en nombre de messages est  $M(N \log N)$  dans tous les cas

Observation : a phase  $i$ , il y a  $(i-1)/2$  processus actif

MU4IN403 – Election de chef

## III.3 Diffusion anonyme

50

### Candidature aléatoire

#### Variables locales

State - Le site est à l'état *active*, *inactive* ou *leader*

Values [N] - Tableau contenant les valeurs émises par tous les sites lors d'une phase

#### Algorithme

State = actif

Repeat

Values = 0;

b = random{0,1}

broadcast(b)

while (timer != 0) do

receive(<b'>) from j

Values [j] = b'

if (b = 1  $\wedge$  Values = 0) then State = leader

if (b = 0  $\wedge$  Values > 0) then State = inactive

until (State != active)

MU4IN403 – Election de chef

## Bibliographie

51

- E. Chang, R.Roberts. *An improved algorithm for decentralized extrema-finding in circular configurations of processes*, Communications of the ACM, ACM, 22 (5): 281–283, 1979.
- H. Garcia-Molina, *Elections in a Distributed Computing System*, IEEE Transactions on Computers, Vol. C-31, No. 1, pages 48–59, 1982.
- D. Hirschberg and J. Sinclair. *Decentralized extrema-finding in circular configurations of processors*. Communications of the ACM, 23 (11): 627–628, 1980.
- A. Itai and M. Rodeh. Symmetry breaking in distributive networks. In Proc. 22nd Annual Symposium on Foundations of Computer Science, FOCS'81, pp. 150{158. IEEE Computer Society, 1981.

MU4IN403 – Election de chef